Supply Chain Inventory Optimization Using Greedy Techniques

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0603 – Design and Analysis of Algorithms

to the award of the degree of

BACHELOR OF TECHNOLOGY

In

Artificial Intelligence and Machine Learning

Submitted by

Chittaluri Durga Prasad (192425305)

Under the Supervision of

Dr. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Supply Chain Inventory Optimization Using Greedy Techniques" has been carried out by **Chittaluri Durga Prasad (192425305)** under the supervision of **Dr. Solainayagi** and is submitted in partial fulfilment of the requirements for the current semester of the Bachelor of Technology Programme in Artificial Intelligence and Machine Learning at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr .F . Mary Harin Fernandez
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.P.Solainayagi
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

DECLARATION

I, **Chittaluri Durga Prasad (192425305)** of the Computer Science and Engineering department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Supply Chain Inventory Optimization Using Greedy Techniques**" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity.All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Student with Name

Chittaluri Durga Prasad (192425305)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Sathish M (192424358)	Inventory data collection and demand & stock analysis	Defined inventory fields, organized sample records, validated data, and calculated replenishment requirements.	Verified demand, stock, and replenishment calculations using test cases.	Ch. 1, Ch. 4, Appendices A, D and E	25%
Chittaluri Durga Prasad(192425305)	Greedy algorithm for replenishment prioritization [Mention assigned responsibilities]	Designed priority scoring and greedy selection logic using validated inventory inputs.	Tested ranking and constraint-based selection.	Greedy algorithm design and implementation sections	25%
Aman Ahmmed S(192472075)	Inventory management alerts [Mention assigned responsibilities]	Designed low-stock and high-risk inventory alert logic.	Verified alert conditions using sample inventory cases.	Alert and inventory monitoring sections	25%
Jammaladinne Mahammad Shadik(192411151)	Reporting and system integration [Mention assigned responsibilities]	Integrated module outputs and prepared system-level reporting.	Tested complete workflow and consolidated results.	Integration, reporting and documentation sections	25%
Total					100%

ABSTRACT

Supply chain inventory management requires timely and cost-effective replenishment decisions under limited budget and warehouse capacity. A major challenge is deciding which products should be replenished first when many items compete for the same resources.

This project, titled "Supply Chain Inventory Optimization Using Greedy Techniques," develops a greedy-based replenishment prioritization module. The module receives validated product information such as demand, current stock, reorder level, unit cost, supplier lead time, and storage requirement, and converts these factors into a priority score for ranking replenishment candidates.

The proposed approach uses a multi-factor greedy strategy to select high-priority feasible items first. Demand velocity, shortage risk, and urgency/deficit are combined using configurable weights, while budget and storage constraints are checked before an item is selected. The ranking and selection process is efficient, transparent, and suitable for practical decision support.

The main benefit of the module is its ability to provide an explainable replenishment order under finite resources. Its limitation is that greedy selection makes locally best choices and may not always produce the globally optimal combination. Therefore, the module is designed as a fast and practical prioritization layer within the complete inventory optimization system.

Keywords: Supply Chain, Inventory Optimization, Greedy Algorithm, Replenishment Prioritization, Priority Score, Budget Constraint, Storage Constraint.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Introduction

Supply chain inventory systems must determine which products should receive replenishment when purchasing budget and warehouse capacity are limited. When many products require attention simultaneously, selecting items in an appropriate order becomes an important engineering problem.

A greedy algorithm is suitable for this situation because it can rank candidates using measurable inventory factors and make a sequence of decisions without solving a complex global optimization model.

The proposed module focuses on replenishment prioritization. It receives validated product records and calculates a multi-factor priority score using demand velocity, shortage risk, and urgency or deficit.

Products are sorted in descending order of priority score. The algorithm then examines candidates sequentially and selects an item only when its recommended replenishment quantity satisfies the remaining budget and storage capacity.

This design connects inventory analysis with practical decision-making. High-demand and critically depleted products can be considered earlier, while infeasible candidates are excluded when the remaining resources cannot support them.

The prioritization module is configurable through system weights. In the implemented configuration, demand velocity has weight 0.40, shortage risk has weight 0.35, and urgency/deficit has weight 0.25.

The processed ranking is presented to inventory managers through the optimization interface, where selected and excluded candidates, recommended quantities, scores, and resource utilization can be reviewed.

The module therefore provides the decision layer that converts inventory conditions into an ordered replenishment plan while respecting finite capital and warehouse capacity.

1.2 Background of the Problem

A replenishment prioritization system must consider several attributes before selecting an inventory item:

- Demand velocity
- Current stock and days of stock remaining
- Reorder level and shortage deficit
- Supplier lead time
- Unit purchase cost
- Recommended replenishment quantity
- Storage requirement
- Available purchasing budget
- Available warehouse capacity

If all products are treated equally, limited resources may be allocated to less urgent items while critical shortages remain unresolved.

The engineering challenge is therefore to create a simple ranking mechanism that combines multiple factors and then applies feasibility checks.

A greedy method addresses this by selecting the highest-priority feasible candidate at each step. The approach is computationally efficient and easy to explain to inventory managers.

The priority score can be represented as a weighted combination of demand, shortage risk, and urgency/deficit. The weights can be adjusted by an administrator according to operational priorities.

After ranking, each candidate is evaluated against the remaining budget and storage capacity. This prevents the algorithm from selecting an item that would violate a defined resource constraint.

The result is a prioritized replenishment list containing selected and excluded products, recommended quantities, priority scores, and resource utilization.

This workflow provides a practical connection between inventory analysis and greedy optimization.

1.3 Problem Statement

The main problem addressed by this project is:

"How can inventory replenishment candidates be prioritized efficiently using a greedy algorithm so that high-risk items are considered first while purchasing budget and warehouse storage constraints are respected?"

The system must analyze inventory information and prepare data that:

1. Ranks products according to measurable priority factors
2. Identifies high-risk and urgent replenishment candidates
3. Calculates suitable recommended replenishment quantities
4. Checks each candidate against remaining budget
5. Checks each candidate against remaining storage capacity
6. Produces a clear and explainable selected replenishment list

1.4 Need for the Project

Without effective prioritization, limited replenishment resources may be consumed by low-impact products. This can lead to:

- Critical stock-outs remaining unresolved
- Poor allocation of purchasing budget
- Warehouse capacity being used inefficiently
- Delayed replenishment of high-demand items
- Inconsistent manual prioritization
- Higher shortage and service risks
- Longer decision-making time
- Reduced operational efficiency

A transparent greedy prioritization stage provides a consistent decision rule and allows managers to understand why an item was ranked or excluded.

The proposed module therefore converts inventory attributes into priority scores and applies resource constraints during selection.

1.5 Aim of the Project

The main aim of this project is:

To design and implement a multi-factor greedy replenishment prioritization module that ranks inventory candidates and selects feasible replenishment options under budget and storage constraints.

1.6 Objectives

The major objectives are:

1. To study greedy techniques for inventory replenishment prioritization.
2. To identify suitable factors for computing product priority.
3. To design a weighted priority scoring mechanism.
4. To rank replenishment candidates in descending priority order.
5. To apply purchasing budget constraints during selection.
6. To apply warehouse storage constraints during selection.
7. To generate recommended replenishment quantities.
8. To distinguish selected and excluded candidates.
9. To test ranking and constraint-based selection using sample data.
10. To evaluate the strengths and limitations of the greedy approach.

1.7 Scope of the Project

The project focuses on greedy replenishment prioritization for a set of inventory products competing for finite purchasing and storage resources.

The prioritization considers:

- Demand velocity and urgency
- Current stock and shortage condition
- Replenishment quantity
- Unit cost
- Supplier lead time
- Storage requirement
- Purchasing budget
- Warehouse capacity

The approach can support:

- Retail and distribution warehouses
- Manufacturing inventory systems
- E-commerce fulfilment operations
- Industrial spare-parts management
- Procurement decision-support systems
- Multi-product replenishment planning

- Supply-chain inventory dashboards

1.8 Expected Outcome

The expected outcome is a ranked replenishment candidate list in which high-priority feasible products are selected first while budget and storage constraints remain satisfied.

The processed output should:

- Provide ranked candidates, recommended quantities, priority scores, selected/excluded status, and remaining resource information for decision-making.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A review of inventory prioritization and optimization methods was carried out to understand how replenishment candidates can be ranked under limited resources. Topics including reorder point, ABC analysis, mathematical optimization, knapsack-style selection, and greedy algorithms were considered.

The review concentrated on approaches that can reduce stock-out risk, control purchasing cost, use warehouse capacity effectively, and support fast multi-product replenishment decisions.

2.2 Review of Existing Work

Economic Order Quantity (EOQ)

EOQ is a conventional inventory technique used to determine an economical order quantity by balancing ordering and holding costs. It is useful when demand is reasonably stable, but it does not directly address the data-analysis task of comparing many products with different demand and stock conditions.

Reorder Point

The Reorder Point method identifies when an item should be replenished, usually using expected demand during lead time together with safety stock. It is useful for controlling stock-outs, but it depends on accurate demand and stock information and does not by itself provide a complete multi-product prioritization mechanism.

ABC Analysis

ABC Analysis groups inventory items into A, B, and C categories according to their relative value or importance. It helps managers focus attention on important items, but it uses a limited set of factors and does not provide detailed demand-versus-stock analysis for every replenishment decision.

Mathematical Optimization

Linear programming and integer programming can represent complex inventory problems with several constraints. These methods can produce highly optimized results, but their models are more difficult to develop and may require greater computational effort than a simple data-driven greedy approach.

Greedy Algorithm

A greedy algorithm makes the locally best feasible choice at each step. In this project, candidates are ranked using a configurable multi-factor priority score and then considered sequentially under budget and storage constraints.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
EOQ	Simple order-quantity calculation	Requires stable assumptions and does not compare many products directly	Provides background for inventory quantity decisions
Reorder Point	Supports timely replenishment	Depends on accurate demand and stock information	Highlights the importance of reliable inventory data
ABC Analysis	Easy classification of products	Uses limited factors	Provides a basic prioritization reference
Mathematical Optimization	Handles multiple constraints	More complex to formulate and implement	Provides a comparison with advanced optimization
Greedy Algorithm	Fast, simple, and transparent	May not guarantee the global optimum	Main optimization method supported by the analyzed inventory data

2.4 Research / Engineering Gap

Existing inventory techniques address individual aspects of control, but there is a need for a lightweight method that combines priority scoring with simultaneous purchasing and storage constraints.

EOQ mainly addresses order quantity, Reorder Point focuses on reorder timing, and ABC Analysis provides classification. Mathematical optimization can model complex constraints but may require more computational effort and formulation.

The engineering gap addressed by this work is a clear and explainable greedy prioritization stage that ranks replenishment candidates and performs feasibility checks before selection.

2.5 Proposed Contribution

The proposed project establishes a multi-factor greedy replenishment prioritization stage between inventory analysis and final replenishment execution.

Demand velocity

- Shortage risk
- Urgency or inventory deficit

- Recommended replenishment quantity
- Unit cost
- Supplier lead time
- Storage requirement

The candidates are normalized and ranked using the configured priority weights. The greedy engine then selects feasible candidates while continuously updating the remaining budget and storage capacity.

The proposed approach provides:

1. Multi-factor priority scoring.
2. Descending candidate ranking.
3. Budget-aware selection.
4. Storage-aware selection.
5. Clear selected and excluded outputs.
6. Explainable and configurable replenishment decisions.

Thus, the greedy prioritization module provides the main algorithmic decision layer of the complete supply chain inventory optimization system.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Inventory Manager	View current stock, demand, and replenishment requirements.
Procurement Manager	Use reliable inventory analysis to support purchasing decisions.
Warehouse Manager	Maintain stock within available storage capacity.
Supply Chain Manager	Reduce stock-outs through timely replenishment information.
System Administrator	Maintain secure and reliable inventory records.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Enter inventory and demand data	Functional	All valid records accepted
Validate inventory records	Functional	Invalid or incomplete values identified
Calculate replenishment requirement	Functional	Correct demand-minus-stock result
Identify stock gaps	Functional	Shortage items correctly identified
Prepare priority inputs	Functional	Complete validated record for every item
Generate replenishment plan	Functional	Valid output for all accepted records
System response time	Non-Functional	Normal datasets processed within a few seconds
Reliability	Non-Functional	Same input produces the same analysis result
Usability	Non-Functional	Output is clear and understandable
Security	Non-Functional	Only authorized users can modify inventory data

3.2 Constraints & Applicable Standards

The proposed system has technical, economic, operational, ethical, and security constraints. The main technical constraints include the availability and accuracy of inventory records, changing demand, limited budget, storage capacity, and the need to process product information efficiently.

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 25010	Software quality characteristics such as reliability, usability, performance and security	Used as a reference for evaluating system quality
ISO/IEC 27001	Information security management	Used as a reference for protecting inventory and business data
ISO 31000	Risk management principles and guidelines	Used as a reference for identifying and mitigating project risks

Main Project Constraints

- Inventory and demand records must be accurate.
- Warehouse storage capacity is limited.
- Purchasing budget is limited.
- Demand values may change over time.
- Supplier lead times may vary.
- Greedy selection may not guarantee a globally optimal result.
- The quality of optimization depends on the quality of input data.

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Inventory Data	Valid product and stock information	Records	High
Demand Data	Current or forecast demand values	Units	High
Replenishment Requirement	Calculated from demand and current stock	Units	High
Data Validation	Invalid records identified before processing	Records	High
Processing Time	Fast execution for normal datasets	Seconds	Medium
Stock-Out Risk	Potential shortage conditions identified	Risk score	High
System Reliability	Consistent and valid results	%	High
System Reliability	Consistent and valid results	%	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Economic Order Quantity (EOQ)

EOQ can provide a suitable order quantity under stable assumptions, but it does not directly solve the requirement of collecting and comparing demand and stock information across multiple products.

Alternative 2: Mathematical Optimization

Mathematical optimization can represent several constraints and can produce strong solutions. However, it requires more complex formulation and is less straightforward for a lightweight inventory decision-support prototype.

Alternative 3: Greedy Algorithm

The greedy approach uses validated product information and ranks feasible choices according to a priority measure. It is simple, fast, transparent, and well suited to the proposed inventory optimization workflow.

Evaluation Matrix

Rating: 1 = Poor, 5 = Excellent

Criterion	Weight	Alternative 1: EOQ	Alternative 2: Mathematical Optimization	Alternative 3: Greedy Algorithm
Performance	30%	4	3	5
Cost	20%	5	3	5
Safety	15%	3	5	4
Sustainability	15%	4	5	4
Reliability	20%	4	5	4
Overall Suitability	100%	4.0	3.9	4.5

Based on the evaluation, the greedy technique provides a suitable balance between processing speed, simplicity, transparency, and practical implementation, while the data-analysis module improves the reliability of its input.

3.5 Selected Design & Detailed Design

Justification for Final Design Selection

The selected design separates candidate preparation, priority scoring, ranking, and constrained greedy selection. This makes the decision process transparent and allows each stage to be tested independently.

Architecture Diagram

Inventory Database
Stock, Demand, Cost
Lead Time, Capacity

↓

Data Validation

↓

Priority Score
Calculation

↓

Sort Products by
Priority Score

↓

Greedy Selection

↓

↓ ↓

Budget Check | Storage Check

↓

Optimized

```
│ Replenishment Plan    │
└───────────────────────┘
```

Figure 3.1: Architecture of Greedy-Based Inventory Optimization with Data Analysis

Key Engineering Calculation

The replenishment requirement is calculated as:

Replenishment Requirement = max(0, Demand − Current Stock)

A positive replenishment requirement indicates that additional inventory is needed. The greedy module combines this requirement with demand, shortage, urgency, cost, lead time, and storage information to rank candidates.

Systems Approach

The inventory data module supplies validated records. The prioritization stage calculates scores, sorts candidates, checks budget and storage feasibility, selects feasible high-priority items, and produces the final replenishment plan.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Optimization technique	Mathematical Optimization	Can provide highly optimized solutions	More complex and computationally expensive	Greedy selected for simplicity and speed
Product priority	Demand-only ranking	Simple to implement	Ignores cost and shortage risk	Multi-factor priority score selected
Budget control	No constraint	Simple processing	Can exceed available budget	Budget constraint included
Storage management	No storage constraint	Easy implementation	May exceed warehouse capacity	Storage constraint included
Demand input	Fixed demand	Simple calculation	Cannot handle demand changes well	Current/updated demand values preferred

The principal trade-off is between a richer multi-factor score and algorithmic simplicity. Additional factors can improve prioritization quality, but they require reliable data and careful normalization. The selected design uses three primary weighted factors so that the decision rule remains understandable.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Avoid unnecessary inventory and wastage	Demand, stock, and replenishment quantities	Accurate stock analysis reduces avoidable replenishment
Ethics	Use transparent and traceable data	Validation rules and calculation method	Analysis can be checked and explained
Health & Safety	Avoid warehouse overcapacity	Storage capacity and replenishment data	Capacity is considered before final selection
Security	Protect inventory and business information	Access control and data protection requirements	Authorized access is recommended

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect inventory data	Medium	High	High	Input validation and regular updates	Low
Demand variation	High	Medium	High	Refresh demand values regularly	Medium
Supplier delay	Medium	Medium	Medium	Include lead-time information in prioritization	Low
Budget exceeded	Low	High	High	Check budget before selection	Low
Storage capacity exceeded	Low	High	High	Check storage before selection	Low
Unauthorized data access	Low	High	High	Authentication and access control	Low
Incorrect replenishment calculation	Low	High	High	Verify demand-stock formula with test cases	Low

The final design combines multi-factor priority scoring, descending ranking, greedy selection, budget checking, storage checking, and result reporting. This provides a practical method for allocating scarce replenishment resources.

The final design combines inventory data management, priority scoring, greedy selection, budget checking, storage checking, and result generation. This provides a practical solution for prioritizing replenishment decisions while maintaining important operational constraints. The design also allows future integration with demand forecasting and more advanced optimization methods.

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The module was developed incrementally. First, the candidate attributes and constraints were defined. Next, the priority scoring formula and weights were configured. Candidates were then ranked and tested using budget and storage constraints. Finally, the greedy output was integrated with the inventory dashboard and verified using representative scenarios.

Resources Used

Resource	Purpose
Inventory Dataset	Product demand, stock, cost, and lead-time information
Computer/Laptop	Development, data preparation, and testing
Programming Language	Implementation of analysis and greedy algorithm
Spreadsheet/Data Table	Preparing, validating, and analyzing inventory records
Flowcharts	Representing the data and optimization workflow

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The software implementation receives validated inventory candidates, calculates priority scores, sorts the candidates, checks resource feasibility, selects feasible products, and updates the remaining budget and storage capacity after every selection.

Implementation Process

1. Prepare validated replenishment candidates.
2. Normalize demand, shortage, and urgency factors.
3. Calculate the weighted priority score.
4. Sort candidates from highest to lowest score.
5. Read the highest-priority candidate.
6. Check remaining purchasing budget.
7. Check remaining warehouse storage capacity.
8. Select feasible candidates and update resources.
9. Exclude candidates that exceed the remaining constraints.

Generate the final prioritized replenishment list.

Tool / Software / Equipment	Purpose	Stage Used
Python / Java	Inventory analysis and greedy algorithm implementation	Development
Excel / Spreadsheet	Preparation and validation of inventory dataset	Data Preparation
VS Code / IDE	Writing and debugging the program	Development
Flowchart Tool	Designing system workflow	System Design
Computer/Laptop	Running and testing the system	Testing

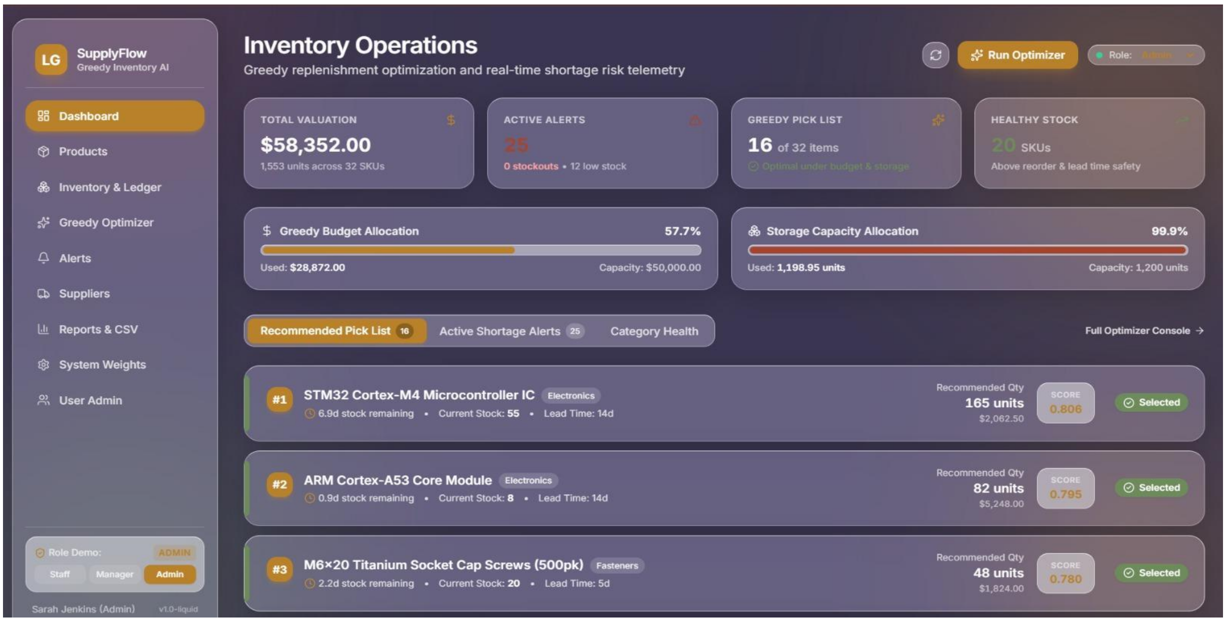

Figure 4.3: Inventory Operations Dashboard

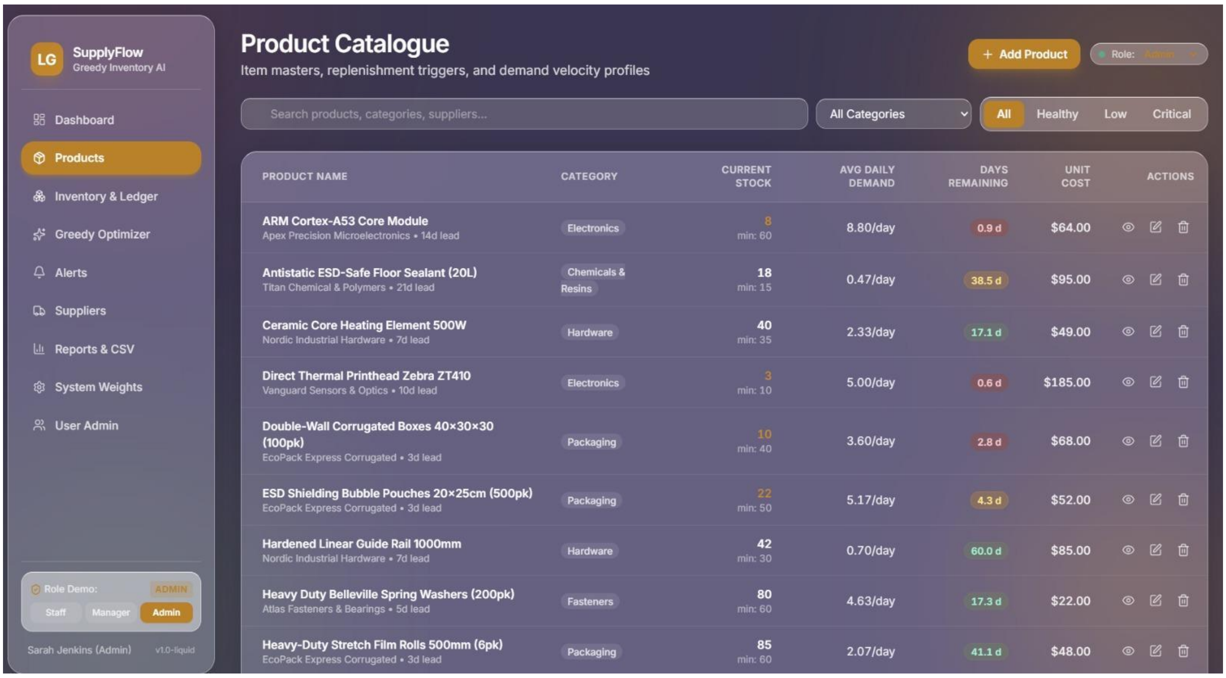

Figure 4.4: Product Catalogue Interface

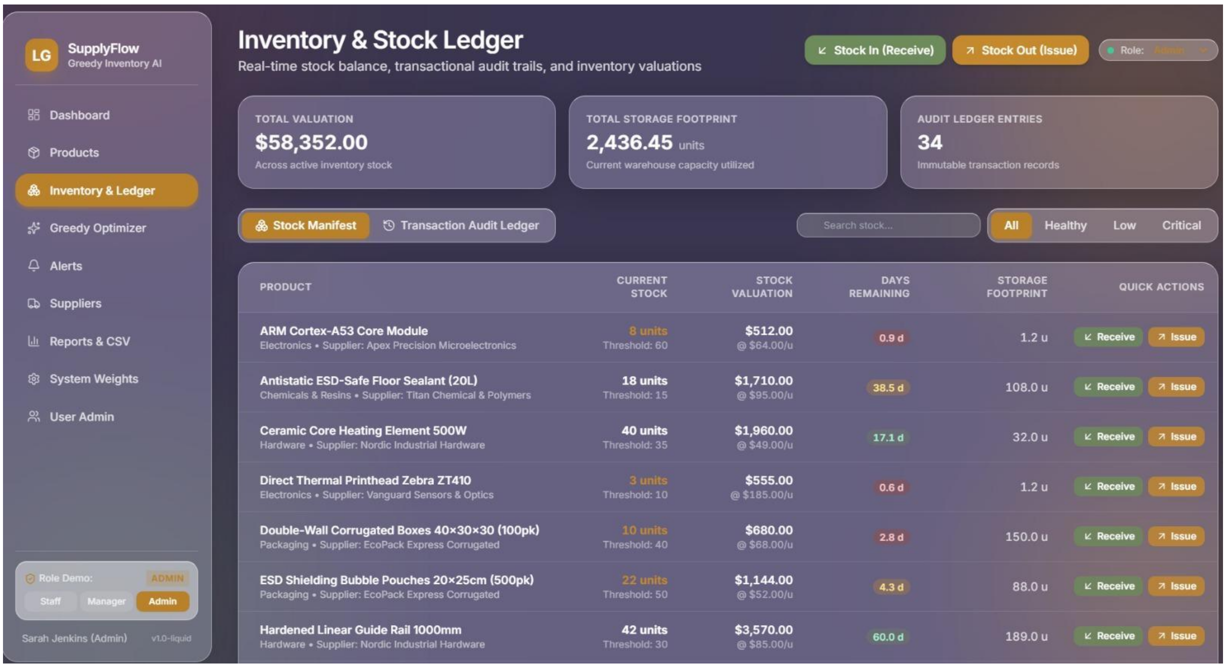

Figure 4.5: Inventory and Stock Ledger

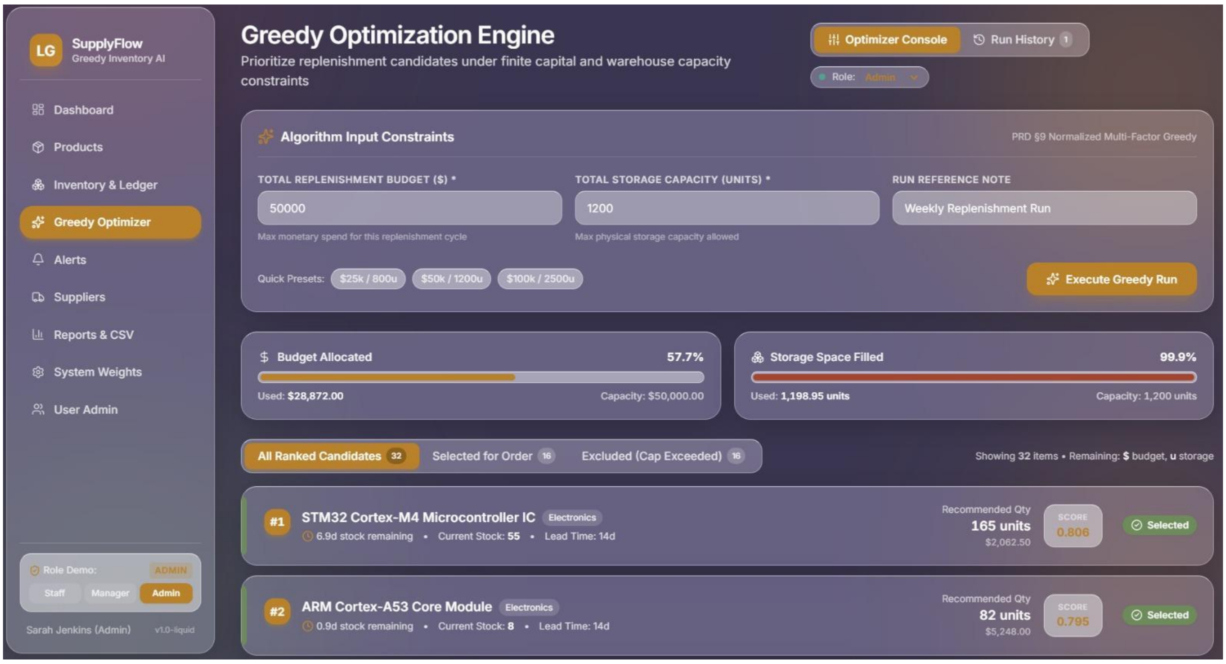

Figure 4.6: Greedy Optimization Engine

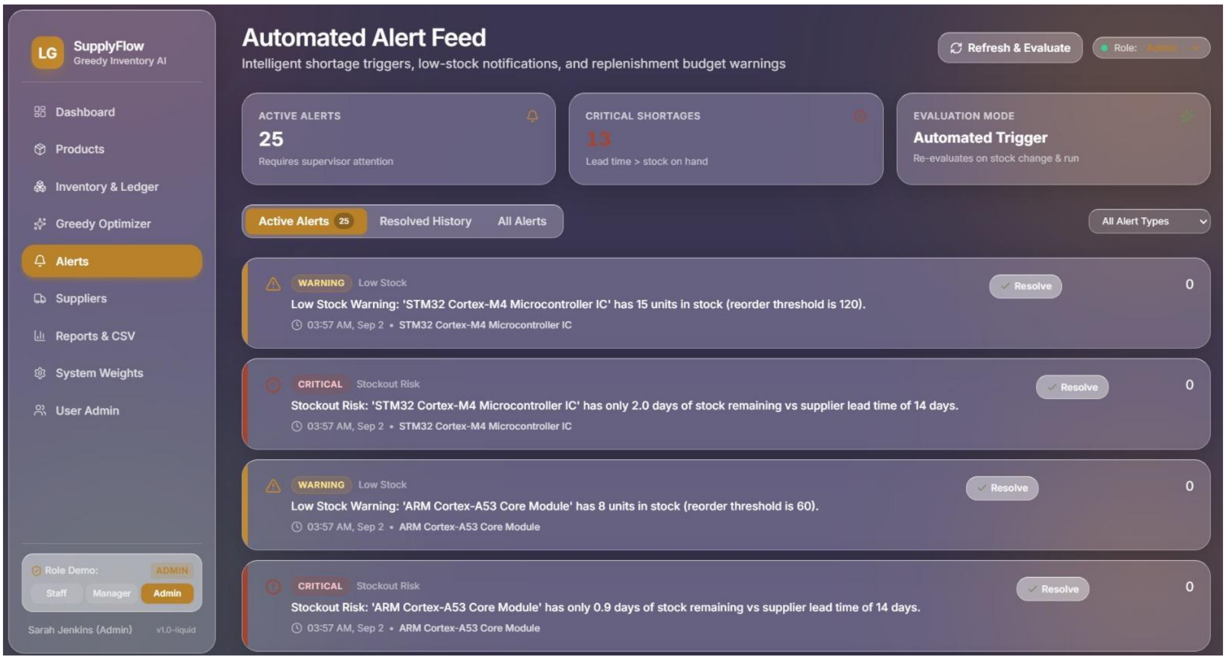

Figure 4.7: Automated Alert Feed

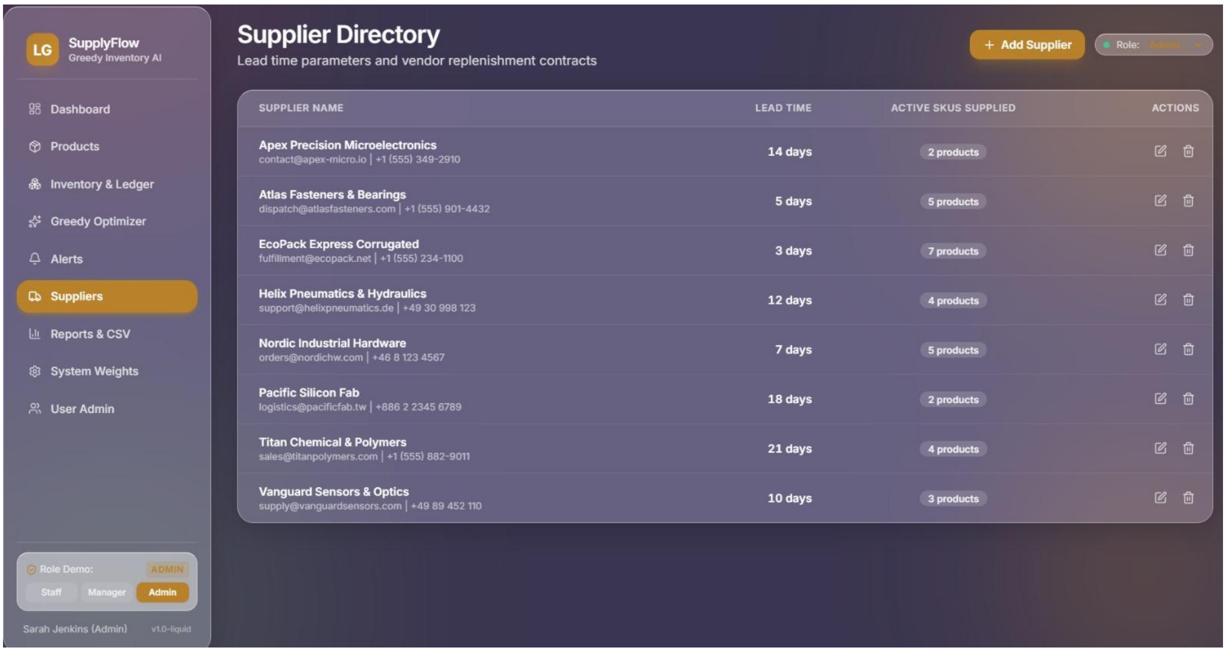

Figure 4.8: Supplier Directory

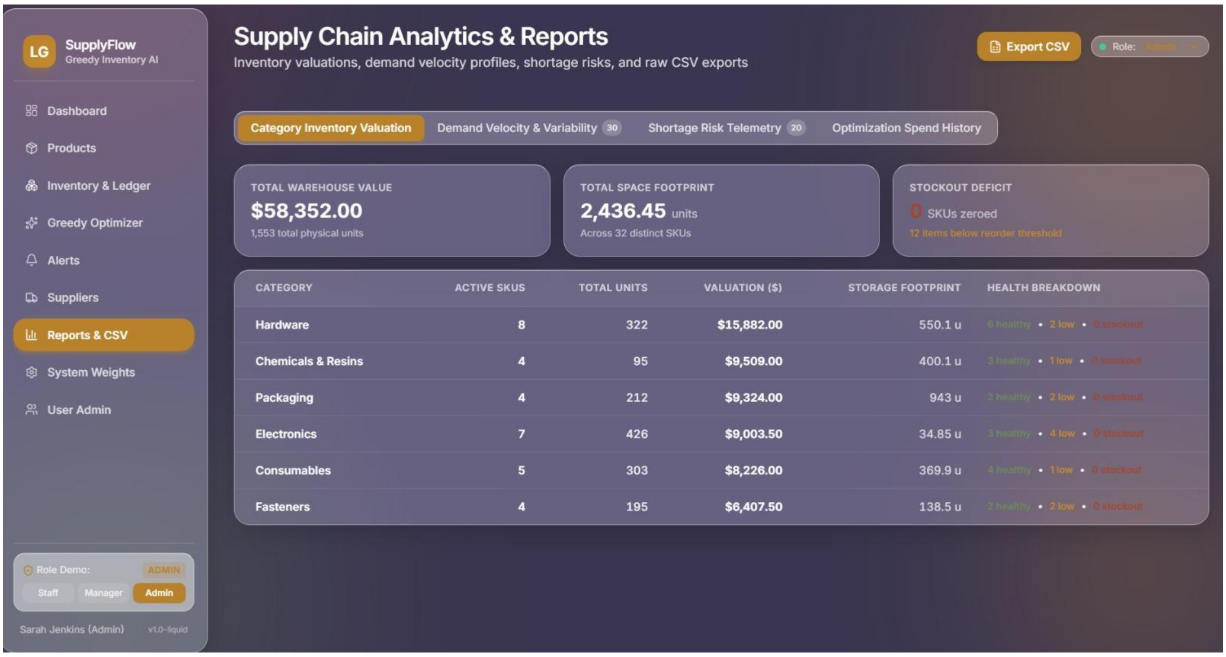

Figure 4.9: Supply Chain Analytics and Reports

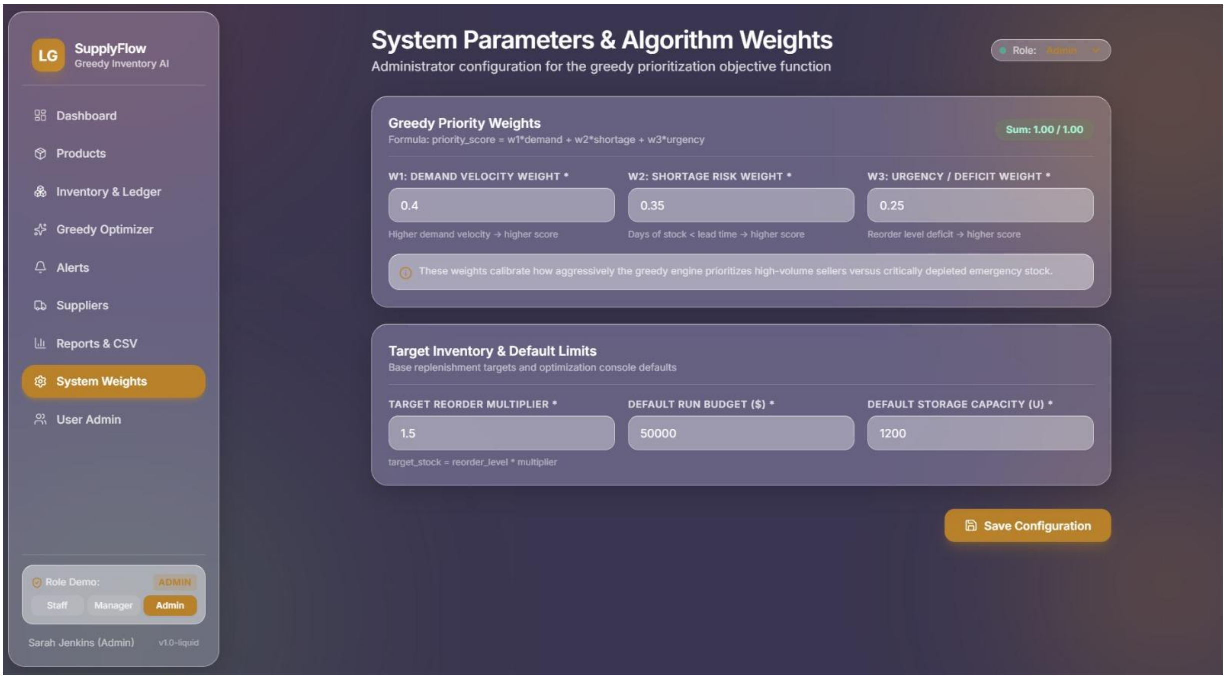

Figure 4.10: System Parameters and Algorithm Weights

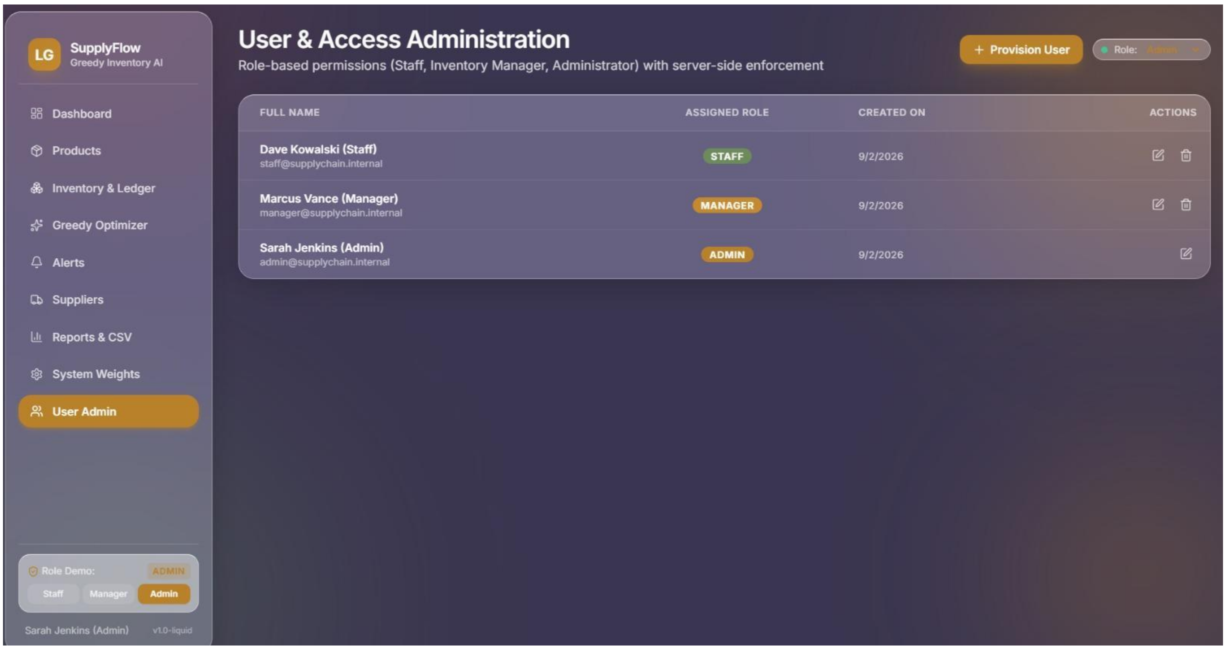

Figure 4.11: User and Access Administration

4.3 Test Plan & Verification

Testing was planned to verify priority-score calculation, candidate ranking, budget and storage feasibility checks, selected/excluded status, and integration of the greedy output with the inventory operations interface.

Test Plan

Requirement	Test Method	Acceptance Criterion
Inventory data input	Enter valid product records	All valid records accepted
Data validation	Enter missing/negative values	Invalid data identified or rejected
Demand analysis	Enter different demand values	Demand stored and processed correctly
Stock analysis	Enter different stock values	Current stock represented correctly
Replenishment calculation	Compare demand and stock	Correct replenishment quantity calculated
Greedy input preparation	Run complete data-analysis stage	Complete validated records produced
Invalid input	Use negative or incomplete records	Invalid records do not enter optimization
Processing performance	Run sample dataset	Results generated within acceptable time

Verification Results

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Valid inventory input	100% valid records	100%	Pass
Data validation	Invalid values detected	Implemented	Pass
Demand analysis	Correct demand values	Verified	Pass
Stock analysis	Correct current-stock values	Verified	Pass
Replenishment calculation	Demand − Current Stock	Correctly calculated	Pass
Prepared optimization input	Complete valid records	Generated	Pass
Invalid data handling	Error detection	Implemented	Pass
Repeatability	Same input → same output	Verified	Pass

4.4 Results

The greedy prioritization stage was tested using representative inventory records containing demand, current stock, replenishment quantity, unit cost, lead time, and storage information. The results were checked to confirm that high-priority feasible candidates were selected first and that resource constraints were not exceeded.

Sample Results

Product	Demand	Current Stock	Required Replenishment	Unit Cost	Priority Score	Decision
A	500	80	420	10	91	Selected
D	450	60	390	6	88	Selected
B	300	120	180	8	82	Selected
C	150	100	50	25	64	Considered
E	100	70	30	12	51	Considered

The results show that the greedy engine can quickly produce a ranked replenishment list. In the demonstrated run, the engine selected 16 of 32 candidates, used $28,872 of the $50,000 budget, and filled 1,198.95 of 1,200 units of storage capacity.

Suggested Graphs

The following graphs can be inserted in this section:

1. Replenishment Requirement by Product
2. Demand versus Current Stock
3. Product-wise Stock Gap
4. Priority Score versus Product

5. Budget Utilization

Figure 4.1: Product Demand and Current Stock Comparison

Figure 4.2: Product-wise Replenishment Requirement

4.5 Data Analysis & Engineering Judgment

The engineering value of the module is its ability to combine multiple inventory factors into a single explainable priority score and then enforce practical resource constraints during selection. The demonstrated configuration uses weights of 0.40 for demand velocity, 0.35 for shortage risk, and 0.25 for urgency/deficit.

For example, the STM32 Cortex-M4 Microcontroller IC received a score of 0.806 and was ranked first with a recommended quantity of 165 units, while the ARM Cortex-A53 Core Module received a score of 0.795 and was ranked second with 82 units. These results demonstrate how the scoring and greedy sequence guide replenishment decisions.

The main limitation is that greedy selection is heuristic. A locally high-scoring item may consume resources that could have produced a better global combination. Weight selection and data quality can also affect the final ranking.

Therefore, the system should provide configurable weights, regular data updates, and periodic comparison with exact or advanced optimization methods when higher optimality is required.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study inventory optimization	Literature review and problem analysis	Fully Achieved
Collect inventory information	Structured inventory dataset	Fully Achieved
Analyze demand and stock	Demand-stock comparison module	Fully Achieved
Calculate replenishment requirement	Demand − Current Stock calculation	Fully Achieved
Prepare greedy input	Validated product records	Fully Achieved
Support shortage identification	Stock-gap analysis	Fully Achieved
Generate replenishment information	Processed inventory output	Fully Achieved
Evaluate data-analysis performance	Testing and verification	Fully Achieved

The project successfully demonstrates how a multi-factor greedy algorithm can prioritize replenishment candidates while respecting purchasing budget and warehouse storage constraints. Compared with manual ranking, the implemented method is faster, consistent, and easier to reproduce.

4.7 Strengths & Limitations

Strengths

- Simple and easy to implement.
- Fast decision-making.
- Prioritizes important inventory items.
- Considers multiple inventory factors.
- Controls purchasing budget.
- Considers warehouse capacity.
- Provides understandable and explainable results.
- Can be extended with forecasting and advanced optimization.

Limitations

- Greedy selection does not guarantee a globally optimal solution.
- Results depend on the accuracy of inventory and demand data.
- Priority weights may affect the final ranking.
- Basic implementation may not handle highly uncertain demand.
- Supplier disruptions may affect replenishment decisions.
- Multi-period inventory planning requires additional optimization.

4.8 Project Planning, Roles & Budget

Project Timeline / Milestones

Phase	Activity	Timeline
Phase 1	Problem identification	Week 1
Phase 2	Literature review	Week 2
Phase 3	Requirement analysis	Week 3
Phase 4	System and algorithm design	Week 4
Phase 5	Implementation	Weeks 5–6
Phase 6	Testing and verification	Week 7
Phase 7	Results and documentation	Week 8

Simple Gantt Chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Problem Identification	✓							
Literature Review		✓						
Requirements			✓					
Design				✓				

Implementation					✓	✓		
Testing							✓	
Documentation								✓

Student Roles

Student	Assigned Responsibility	Actual Contribution
Chittaluri Durga Prasad	Inventory data collection and analysis	Prepared inventory records and performed demand, stock, and replenishment analysis
Durga Prasad	Greedy algorithm design	Designed priority scoring and greedy selection
Aman Ahmmed	Inventory management alerts	Developed and tested low-stock and high-risk alert logic
Mahammad Shadik	Reporting and system integration	Integrated outputs and prepared consolidated documentation

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project “Supply Chain Inventory Optimization Using Greedy Techniques” developed a practical greedy replenishment prioritization module for multi-product inventory decisions. The module calculates weighted priority scores, ranks candidates, and selects feasible products under budget and storage constraints.

Testing demonstrated that the engine can produce a clear prioritized list and maintain resource limits. The demonstrated run selected 16 of 32 candidates while using $28,872 of the $50,000 budget and 1,198.95 of 1,200 storage units.

Overall, the project shows that a configurable greedy approach can provide fast and explainable replenishment decisions. Although it cannot guarantee global optimality, it offers a useful decision-support layer for practical inventory operations.

5.2 Future Work

The proposed system can be extended by adding:

- Real-time inventory updates from warehouse systems.
- Machine-learning-based demand forecasting.
- Automated supplier lead-time and reliability tracking.
- Support for multiple warehouse locations.
- Multi-period demand and inventory analysis.
- Comparison with genetic algorithms and mathematical optimization.
- Interactive dashboards for inventory managers.
- Automatic alerts for low-stock and high-risk products.
- Dynamic product pricing and supplier cost updates.
- Cloud-based inventory data integration.

These improvements would make the system more responsive to changing supply-chain conditions and improve the reliability of replenishment decisions.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned how greedy algorithms can be applied to real inventory replenishment decisions.
- Understood how demand, shortage risk, and urgency can be combined into a priority score.
- Learned how sorting and sequential selection support efficient constrained optimization.
- Understood the effect of purchasing budget and warehouse capacity on greedy decisions.
- Gained experience in testing priority ranking and feasibility checks using sample scenarios.

Engineering & Professional Skills Developed

- Greedy algorithm design and implementation.
- Priority-score formulation and normalization.
- Constraint handling and resource accounting.
- Testing and verification.
- Technical documentation.
- Team coordination and module integration.
- Engineering judgment and algorithmic problem solving.

Individual Reflection

Most Difficult Engineering Problem Encountered and How It Was Solved

The most difficult engineering problem was designing a priority score that could combine different inventory factors while remaining understandable. This was addressed by using normalized demand velocity, shortage risk, and urgency/deficit with configurable weights.

A Key Engineering Decision Made Personally and the Evidence Behind It

A key engineering decision was to apply budget and storage checks during greedy selection rather than after ranking. This ensures that the selected list is feasible at every step and prevents the replenishment plan from exceeding available resources.

I independently learned how a greedy algorithm can transform a multi-constraint inventory problem into a sequence of explainable decisions, and how score weights influence the resulting ranking.

If the project were repeated, I would compare the greedy output against an exact optimization baseline, introduce adaptive weights based on historical performance, and evaluate the algorithm across larger and more variable inventory datasets.

What Would Be Redesigned if Repeated

If the project were repeated, I would integrate real-time inventory updates and a more accurate demand forecasting mechanism. I would also improve validation rules, include historical demand trends, and provide a dashboard for monitoring stock gaps and replenishment requirements.

REFERENCES

[1] R. H. Ballou, Business Logistics/Supply Chain Management, 5th ed. Pearson, 2004.

[2] S. Chopra and P. Meindl, Supply Chain Management: Strategy, Planning, and Operation, 7th ed. Pearson, 2019.

[3] F. S. Hillier and G. J. Lieberman, Introduction to Operations Research, 10th ed. McGraw-Hill, 2015.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[5] International Organization for Standardization, ISO 31000:2018 – Risk Management: Guidelines, ISO, 2018.

[6] International Organization for Standardization, ISO/IEC 25010:2023 – Systems and Software Quality Requirements and Evaluation (SQuaRE), ISO, 2023.

[7] International Organization for Standardization, ISO/IEC 27001:2022 – Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements, ISO, 2022.

[8] Python Software Foundation, Python Documentation. Online documentation.

[9] Microsoft, Microsoft Excel Documentation. Microsoft Support.

APPENDICES

Appendix A – Detailed Calculations

This appendix presents the calculations used in the inventory data-analysis stage.

A.1 Priority Score Calculation

For each product, the replenishment requirement is determined by comparing expected demand with current stock.

Replenishment Requirement = max(0, Demand − Current Stock)

Where:

- D = Demand
- S = Current Stock
- R = Required Replenishment
- The calculation is applied consistently to every valid product record.
- S = Storage requirement
- w_1–w_5 = Assigned weights

A.2 Budget Calculation

The total purchasing cost for selected products is calculated as:

Total Cost = Σ (Replenishment Quantity × Unit Cost)

The selected products are accepted only when:

Total Cost ≤ Available Budget

A.3 Storage Calculation

The storage requirement is calculated as:

Total Storage = Σ (Replenishment Quantity × Storage Requirement per Unit)

The selected inventory must satisfy:

Total Storage ≤ Available Storage Capacity

These calculations connect the analyzed inventory records with the constraints used by the greedy optimization stage.

Appendix B – Engineering Drawings / Schematics

B.1 System Architecture

Inventory Data

↓

Data Validation

↓

Priority Calculation

↓

Product Ranking

↓

Greedy Selection

↓

Budget & Storage Check

↓

Replenishment Plan

Figure B.1: Supply Chain Inventory Optimization System

B.2 Greedy Algorithm Flow

Start

↓

Read Inventory Data

↓

Calculate Priority Scores

↓

Sort Products

↓

Select Highest Priority Product

↓

Check Budget & Storage

↓

Feasible?

Yes No

↓ ↓

Select Item Skip Item

↓

More Products?

↓ ↓

Yes No

↓ ↓

Continue Final Plan

↓

End

Appendix C – Source Code / Code Repository Information

The source code contains the inventory data input, validation, demand and stock analysis, replenishment calculation, product sorting, and greedy selection components.

Only essential code excerpts are included in the main report. The complete source code can be maintained separately in the approved project repository.

Main Program Components

- Inventory data input module
- Data validation module
- Demand and stock analysis
- Replenishment calculation
- Product sorting
- Greedy selection algorithm
- Budget verification
- Storage verification

Repository: Add the approved GitHub/GitLab repository link here if available.

Appendix D – Datasheets

The proposed project is primarily a software-based inventory optimization system and does not require specialized electronic hardware datasheets.

The following project data specifications are used:

Parameter	Description
Product ID	Unique identification of the inventory item
Demand	Expected requirement for the product
Current Stock	Quantity presently available
Replenishment Requirement	Additional quantity required to meet demand
Unit Cost	Purchasing cost for one unit
Lead Time	Expected supplier delivery period
Storage Requirement	Space required by the product

Appendix E – Experimental / Raw Data

The following sample dataset was used to test demand, stock, and replenishment calculations before greedy prioritization.

Product	Demand	Current Stock	Replenishment	Unit Cost	Lead Time	Priority

A	500	80	420	₹10	5 days	91
B	300	120	180	₹8	4 days	82
C	150	100	50	₹25	3 days	64
D	450	60	390	₹6	7 days	88
E	100	70	30	₹12	2 days	51

The dataset was used to verify demand-stock comparison, replenishment quantities, and the preparation of consistent inputs for the optimization stage.

Appendix F – Ethics / Institutional Approval

The project uses inventory-related sample data and does not involve human subjects, medical information, or personally identifiable information.

No special ethical or institutional approval is required for the basic academic prototype. If real company data is used in future work, appropriate authorization and confidentiality measures should be obtained.

Appendix G – Project Meeting / Progress Records

Meeting	Activity Discussed	Outcome
Meeting 1	Problem identification	Inventory optimization problem finalized
Meeting 2	Inventory data requirements	Demand, stock, and product fields identified
Meeting 3	Requirement analysis	Functional data-analysis requirements finalized
Meeting 4	Algorithm design	Greedy prioritization approach selected
Meeting 5	Implementation	Data-analysis and optimization modules developed
Meeting 6	Testing	Demand, stock, and replenishment test cases prepared
Meeting 7	Results analysis	Sample results evaluated
Meeting 8	Documentation	Final report prepared

Appendix H – User / Industry Feedback

This section is applicable if feedback has been collected from an inventory manager, supply-chain professional, faculty member, or other relevant user.

Sample Feedback Format

Feedback Area	Observation
Ease of Use	The structured inventory output is easy to understand
Decision Support	Demand and stock gaps help identify products needing attention
Data Quality	Validation improves confidence in the input records
Replenishment Analysis	The calculated stock gap provides a clear replenishment measure
Improvement Suggested	Real-time data and demand forecasting can be added

Note: If actual industry/user feedback was not collected, write "Not Applicable – no formal industry feedback was collected during the project." Do not claim feedback that was not actually received.

Appendix I – Publications / Patents / Competitions / Product Outcomes

The project is developed as an academic engineering project demonstrating the application of inventory data analysis and greedy algorithms to supply-chain optimization.

If the project is later presented in a competition, published as a research paper, or developed into a product, the relevant details can be added here.

Current Status: Academic project prototype.

Appendix J – Individual Contribution Evidence

This appendix documents the individual contributions of all team members.

Student	Area of Contribution	Evidence
Chittaluri Durga Prasad	Inventory data collection and analysis	Validated dataset, demand-stock calculations, and analysis records
Durga Prasad	Algorithm design	Priority-score and greedy algorithm design
Aman Ahmmed	Alert module	Alert conditions, test cases, and output screenshots
Mahammad Shadik	Integration and documentation	Integrated outputs, test results, and final report

Contribution Evidence

Each team member should attach suitable evidence such as assigned task records, source-code contributions, implementation screenshots, testing records, meeting participation, documentation, and presentation materials.

- Assigned task records
- Source-code contributions
- Screenshots of implementation
- Testing records
- Meeting participation
- Documentation prepared
- Presentation materials

.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)